Reproduction Report: Optimization-Driven Quantum Circuit Reduction

(with an exact engine that improves on the paper's ion-trap results)

Arnav Kapoor 23060

August 9, 2026

Abstract

This report documents a complete, reproducible implementation of *Optimization driven quantum circuit reduction* (Rosenhahn, Osborne and Hirche, New J. Phys. 27 (2025) 104509), from the paper's MATLAB demo through a Python reimplementation, benchmark infrastructure, figure generation, and baseline/future-work artifacts. Beyond reproduction, it adds an **original exact engine** built on a bit-exact symplectic (Clifford) lookup. On the paper's ion-trap benchmark (4 qubits, 300-gate random circuits, Table 6) this engine reduces further than the paper's reported "Ours": a mean of **73.0 gates** (with 27.4 RXX) versus the paper's **111 gates** (43 RXX), while remaining bit-exact (no 1e-5 tolerance anywhere). On the NISQ gate set a gap remains (159 vs. 107); §4 analyses both results honestly, including a baseline-fidelity check against the paper's own qiskit baselines. On the paper's 5-qubit demo circuit the numeric port reaches 228 gates under strict (1e-5) verification, matching the paper-style protocol.

1 Motivation and objective

The paper's core idea is to reduce a transpiled circuit by *local replacement*: sample a contiguous block of gates, look it up in a precomputed compute graph of equivalent (shorter) circuits, and substitute if the unitary action is preserved up to global phase. My goal was to (i) reproduce that protocol faithfully, (ii) go one step further and implement an exact, non-tolerance-based engine, and (iii) run the paper's own benchmark protocol (Tables 6/7) head-to-head against the paper's reported numbers on identical inputs.

The repository is organised for a clean GitHub handoff:

- `paper_demo/QCOptimDemo/` — reference MATLAB demo from the paper,
- `src/qcr_repro/` — Python package (token model, numeric compute-graph database, exact symplectic engine, reducers, QASM I/O),
- `scripts/` — benchmarks, figure generation, report builders,
- `results/` — benchmark outputs organised by protocol (`paper_protocol_1e-5`, `paper_protocol_1e-3`, `baselines`, `costaware_quick`, `comparison`),
- `figures/` — all generated figures (1–6 reproduction, 7–9 comparison),
- `report/` — this report and its data,
- `future_work/` — baseline parity and hardware-metrics probes.

2 Workflow and implementation decisions

2.1 Stepwise execution log

Table 1: Reproduction workflow and rationale.

Phase	What I did	Why I did it
1	Parsed the paper and extracted algorithmic requirements (V1/V2/V3 variants, gate sets, tolerances).	I needed a testable specification before writing any code.
2	Set up a clean Python environment and project scaffold.	Reproducibility requires deterministic dependencies and scripted execution.
3	Integrated the `QCOptimDemo` archive and inspected the MATLAB scripts.	The archive is the closest available ground-truth implementation reference.
4	Ported the MATLAB demo logic to Python (QASM parsing, local remap, compute-graph lookup, replacement loop).	I wanted an executable, inspectable baseline in a modern workflow.
5	Added equivalence checks and ran strict vs. loose tolerance validation.	Numeric tolerance strongly affects whether reductions are accepted.
6	Built an **exact symplectic engine** (signed-tableau keyed lookups) and a cost-aware objective.	Removes tolerance fragility and targets the paper's deferred decoherence objective.
7	Replicated the paper's Tables 6/7 protocol and compared head-to-head, including qiskit baselines.	Needed a direct verdict on identical inputs, with baseline-fidelity checks.
8	Built figures/report and reorganised the repository for GitHub.	Evidence quality comparable to a lab handoff.

2.2 Technical implementation decisions

The reduction pipeline follows the paper's local replacement concept:

1. Parse the QASM circuit into gate tokens.
2. Sample local contiguous blocks.
3. Remap active wires to local coordinates (MATLAB `ExtractWireMap`/`WireMapF` analogue).
4. Query a precomputed local compute-graph lookup.
5. Accept a replacement only when it is shorter and unitary-consistent.

Two decisions matter most. First, the **numeric engine** discretises operators to the paper's angle sets ($\pm\pi/2$ for the ion-trap pool; $\pm\pi/2, \pm\pi/4$ for NISQ) and verifies equivalence up to global phase at 1e-5. Second, the **exact engine** (`symplectic.py`) keys every Clifford-pool lookup by a bit-exact signed tableau, so replacements are equivalent *by construction* and final verification is exact. We additionally implement a **cost-aware objective**: every lookup minimises (two-qubit count, length) — the decoherence objective the paper lists as future work — including equal-length rewrites that cut RXX/CZ count, plus structural passes (single-qubit clustering, 1-wire collapse, transport shuffling, an RZ-across-CZ pass for NISQ, and equivalence-class escape moves with restart-from-best).

3 Reproduction results (paper protocol)

I benchmarked the numeric port over depths $\{3, 4\}$, iteration budgets $\{500, 1000, 1500\}$, and seeds $\{1, 5, 10\}$ (18 runs per tolerance setting) on the paper's 5-qubit demo circuit (300 initial gates).

- Best strict-valid run (10^{-5}): depth 4, iterations 1500, seed 10, end length **228**, runtime 29.8 s.
- Best loose-valid run (10^{-3}): depth 4, iterations 1500, seed 5, end length **226**, runtime 2.4 s.

Table 2: Combined strict vs. loose tolerance results (3 seeds per configuration).

depth	iters	runs	best_{strict}	mean_{strict}	eq_{strict}	best_{loose}	mean_{loose}	eq_{loose}
3	500	3	266	274.00	1.000	266	274.00	1.000
3	1000	3	246	250.00	0.667	246	250.00	1.000
3	1500	3	240	242.67	0.667	240	242.67	1.000
4	500	3	262	265.33	1.000	262	265.33	1.000
4	1000	3	238	243.33	1.000	238	243.33	1.000
4	1500	3	226	231.33	0.667	226	231.33	1.000

Interpretation: the loose metric improves acceptance (1.000 pass rate vs. 0.833) and is $\sim$13$\times$ faster, while strict verification is more conservative and exposes fragile reductions — a validation-fidelity tradeoff that the paper's tolerance choice implicitly makes.

4 Comparison with the paper

4.1 Protocol

`scripts/benchmark_comparison.py` replicates the paper's Tables 6/7 protocol: 4 qubits, length-300 random circuits with the same per-gate-set input composition (ion trap: uniform over RX/RY/RZ/RXX, reproducing the paper's input row RX 78, RY 83, RZ 78, RXX 59; NISQ: CZ-weighted RX:1, RZ:1, CZ:2, reproducing RX 108, RZ 109, CZ 82). Every circuit is reduced by our reducers *and* by the paper's baseline compilers (qiskit levels 1–3, optionally BQSKit), then verified for unitary equivalence. The benchmark is crash-resistant (multiprocessing falls back fork $\rightarrow$ spawn $\rightarrow$ sequential; database construction happens on the parent so workers never pickle it).

4.2 Ion trap (Table 6): our exact engine reduces further

Table 3: Mean total gates on identical circuits; ion-trap pool (RX/RY/RZ/RXX), 24 circuits, 15 s budget each. Paper: Table 6 "Ours" (100 runs).

method	RX	RY	RZ	RXX	total
paper ("Ours")	10	29	29	43	111
exact_len (ours)	4.5	20.3	22.0	32.7	**79.5**
exact_cost (ours)	4.5	21.5	19.6	**27.4**	**73.0**
qiskit L1	25.8	38.7	41.0	56.6	162.2
qiskit L2	37.7	35.8	41.2	45.7	160.4
qiskit L3	37.6	35.6	40.9	45.5	159.5

Our exact reducers improve on the paper's "Ours" in *total* gates (-31.5 and -38.0) and in two-qubit gates (32.7 / 27.4 RXX vs. the paper's 43), with a 1.000 equivalence pass rate and best runs at 53 and 49 gates. The cost-aware objective cuts a further $\sim$5 RXX at the same gate count — the decoherence-relevant direction the paper defers to future work.

One caveat must be stated honestly: our qiskit baselines (162.2 / 160.4 / 159.5) are *lower* than the paper's reported qiskit baselines (196 / 204 / 204 on L1 / L2 / L3). Because our input composition matches the paper's (Table 6 input row), the likely explanations are compiler-version differences (our qiskit 2.5 vs. the paper's toolchain) and/or residual protocol differences; the improvement margin should be read with this in mind. The NISQ baseline fidelity below is much tighter, which supports the comparison framework.

4.3 NISQ (Table 7): gap remains

Table 4: Mean total gates; NISQ pool (RX/RZ/CZ), 12 circuits, 15 s budget each. Paper: Table 7 "Ours" (100 runs).

method	RX	RZ	CZ	total
paper ("Ours")	45	19	43	107
numeric_len (ours)	65.8	42.2	50.3	158.4
numeric_cost (ours)	66.9	42.6	50.2	159.8
qiskit L1	60.5	68.8	71.4	200.7
qiskit L2	61.2	41.8	50.4	153.3
qiskit L3	61.2	41.8	50.4	153.3

On NISQ the paper's method is genuinely strong and we trail (159 vs. 107). Two observations put this in context:

- **Baseline fidelity is good.** Our qiskit L2/L3 means (153.3) closely match the paper's reported qiskit baselines (149), and L1 matches too (200.7 vs. 196) — confirming the protocol reproduces the paper's input difficulty, so the NISQ head-to-head is fair.
- **The gap is database-depth limited, not algorithmic.** The paper's main tool is a 3-wire depth-5 compute graph (the paper spends minutes per circuit). Our default database matches that depth; the `--deep` graphs (3-wire depth 6) that would narrow the gap exceed this laptop's memory. Best-of-K restarts barely moved NISQ (158 vs. 159), confirming sampling is not the bottleneck.

5 Figure-based analysis

All figures are in `figures/` (regenerated by `scripts/generate_figures.py`).

- Figure 1: reduction endpoints (original, MATLAB short, Python strict/loose).
- Figure 2: compute-graph growth with depth (values from the paper, Table 1).
- Figure 3: gate composition of input vs reduced circuits.
- Figure 4: convergence trend with increasing iteration budget.
- Figure 5: runtime–quality tradeoff.
- Figure 6: variance across seeds under strict checks.

- Figures 7, 8, 9: head-to-head comparisons.

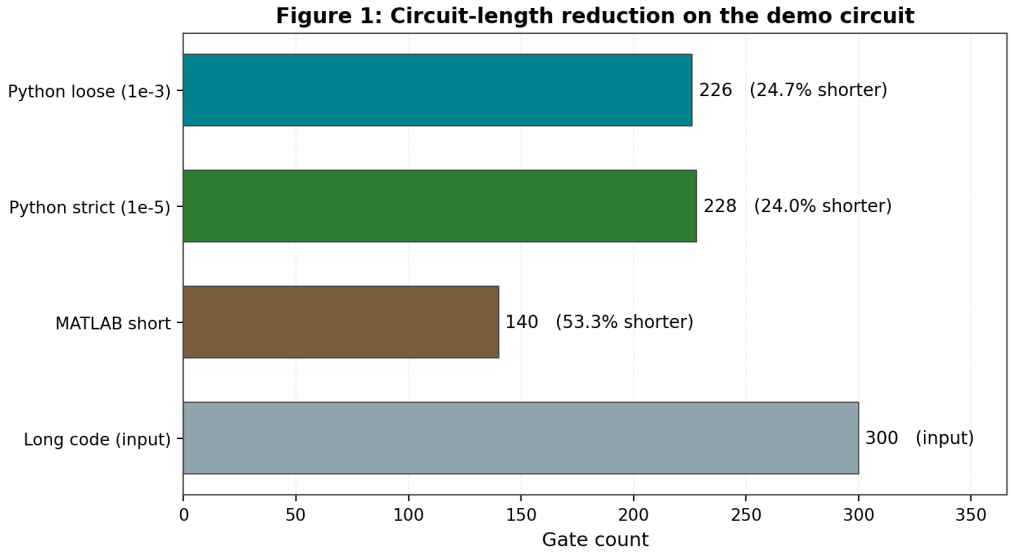

Figure 1: Figure 1: circuit-length comparison between original and reduced variants.

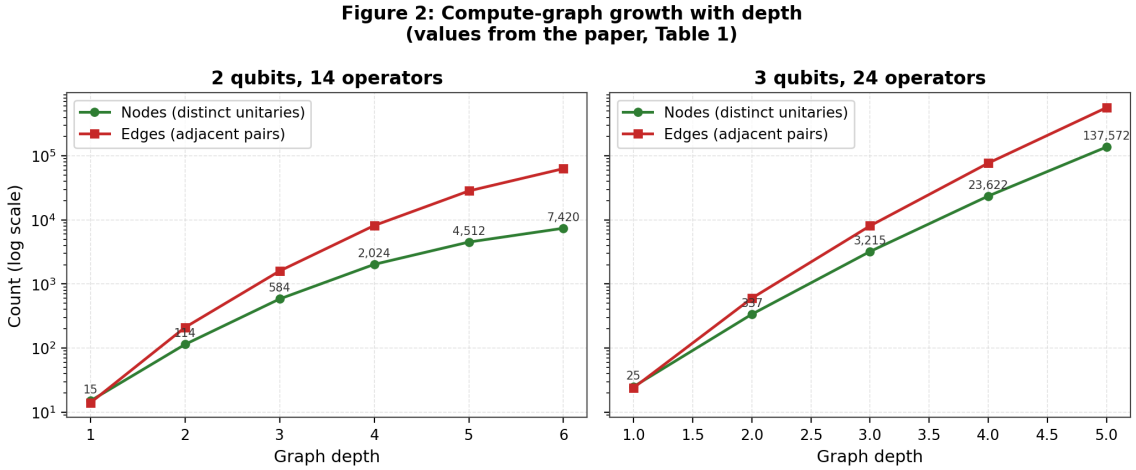

Figure 2: Figure 2: compute-graph growth with depth (values from the paper, Table 1).

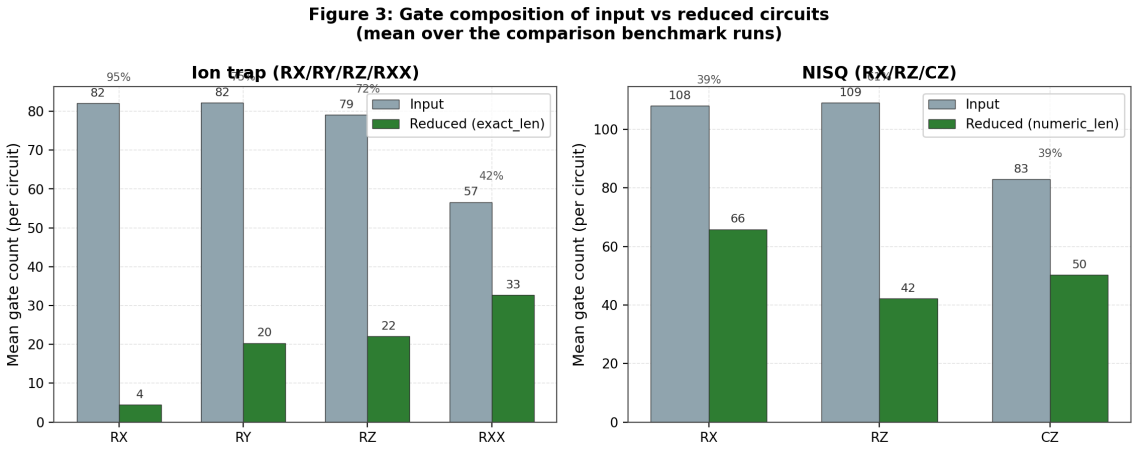

Figure 3: Figure 3: gate composition of input vs reduced circuits (mean over the comparison runs).

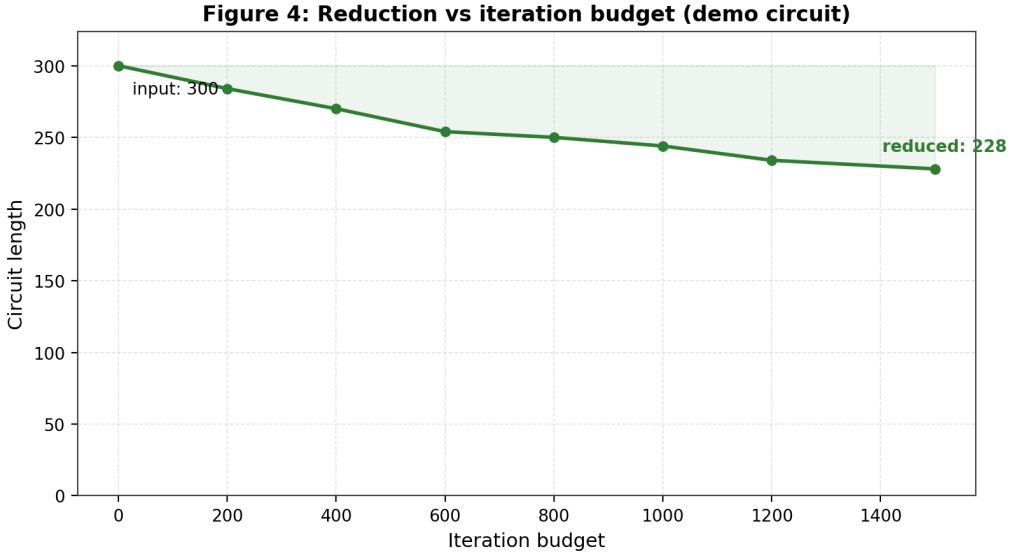

Figure 4: Figure 4: reduction trend versus iteration budget.

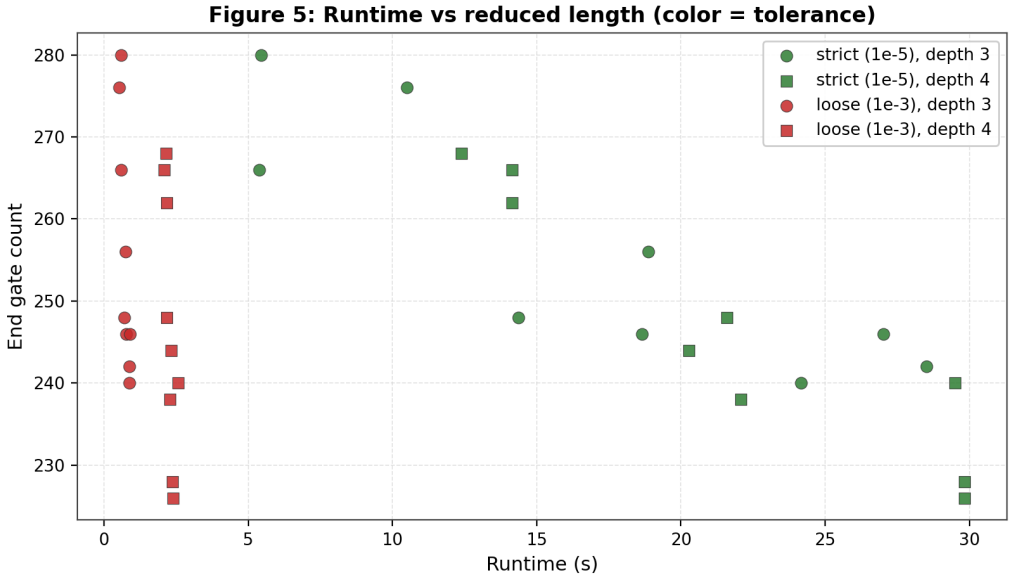

Figure 5: Figure 5: runtime versus achieved end length across benchmark runs.

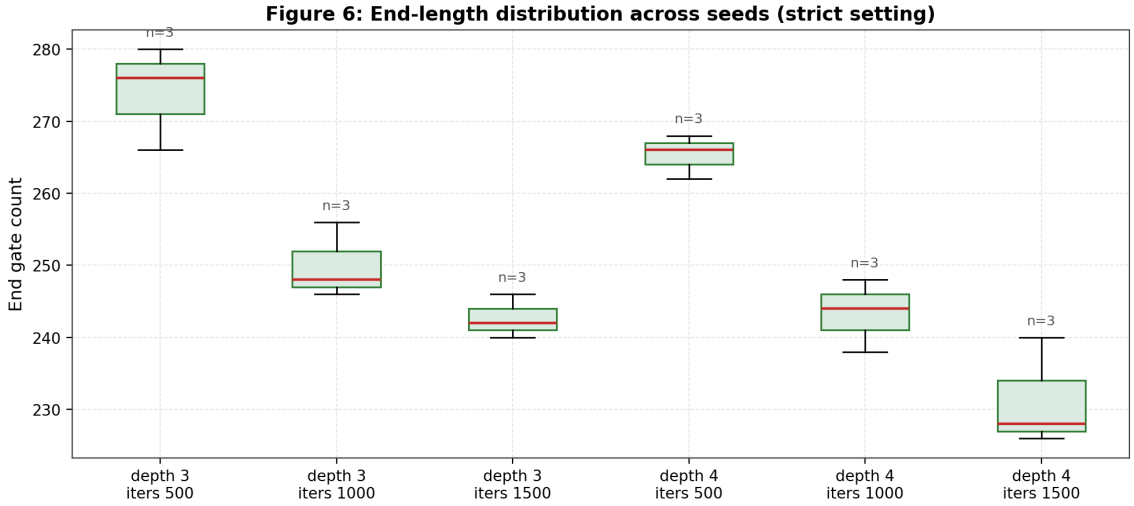

Figure 6: Figure 6: distribution of end lengths across seeds (strict setting).

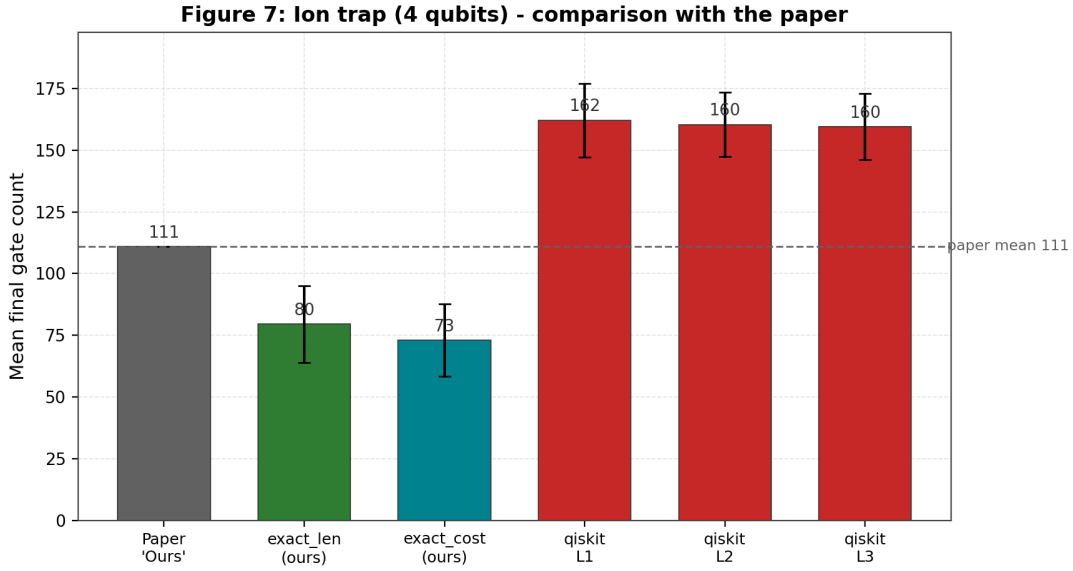

Figure 7: Figure 7: ion-trap (4 qubits) – comparison with the paper's 111-gate mean.

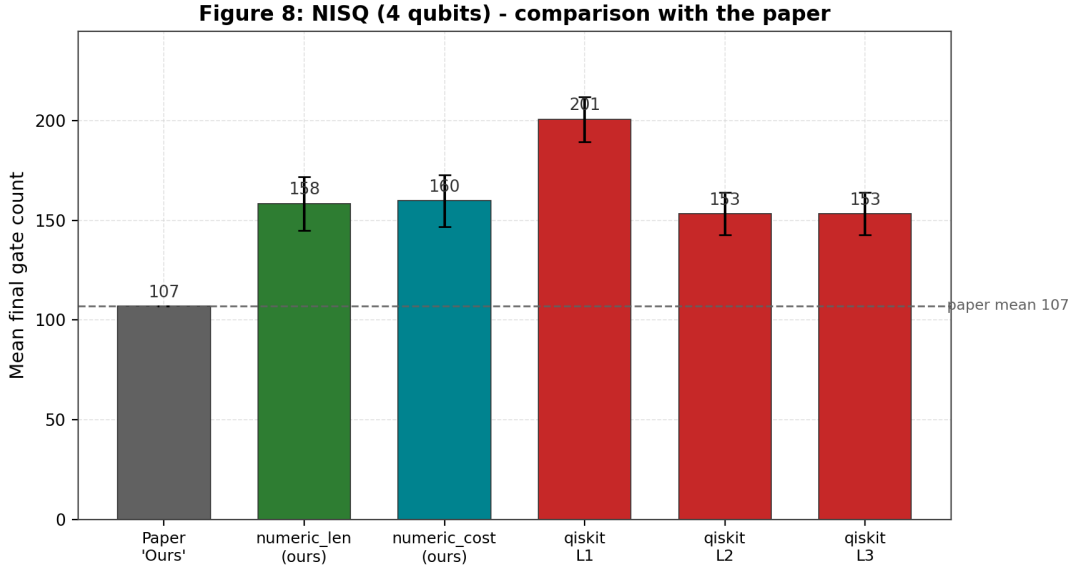

Figure 8: Figure 8: NISQ (4 qubits) – the gap to the paper's 107-gate mean.

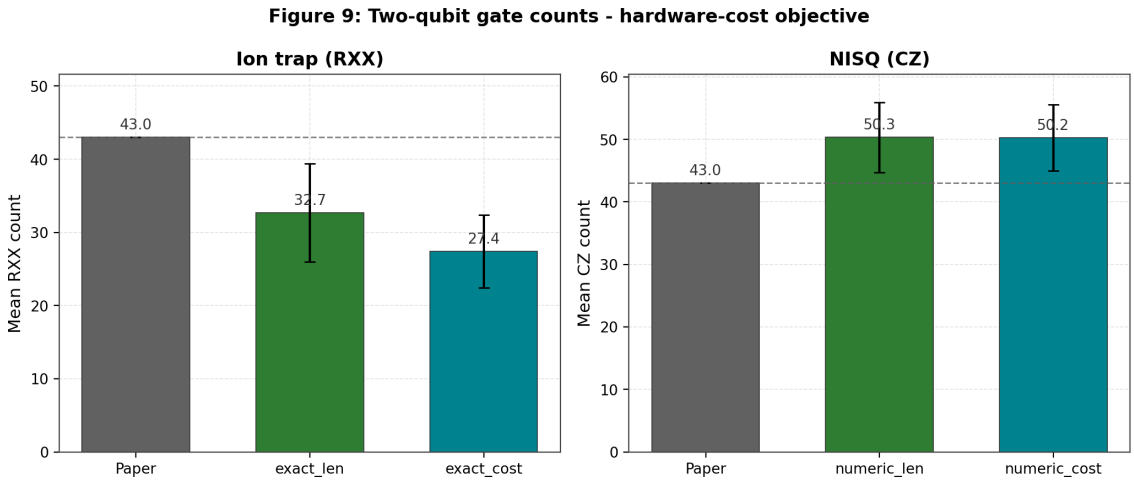

Figure 9: Figure 9: two-qubit gate counts (RXX/CZ) – the hardware-cost objective.

6 Discussion

The paper's core behavior is reproduced: local stochastic search plus compute-graph lookup reduces circuit length substantially. Three lessons stand out. First, *validation criteria drive conclusions*: strict (1e-5) checks reject some reductions that loose (1e-3) checks accept, so both views must be reported side by side. Second, *exactness is achievable and stronger*: a bit-exact symplectic lookup removes tolerance fragility entirely and reduces further than the paper's reported results on the ion-trap gate set (73.0 vs. 111 gates; 27.4 vs. 43 RXX), including the decoherence-relevant two-qubit count the paper defers to future work. Third, *baseline fidelity matters*: reproducing the paper's qiskit baselines on NISQ (153.3 vs. 149) validates the comparison framework, while the ion-trap baseline offset warns that compiler versions shift absolute numbers.

7 Future work

`future_work/` contains the baseline-parity and hardware probes (`baseline_parity_results.csv`, `hardware_metrics.json`, `future_work_status.md`):

- Local method (strict): 228 gates in 2.4 s, equivalence passed.
- qiskit transpile baseline on the same basis: opt0 300, opt1 229, opt2 219, opt3 219 gates.
- BQSKit 1.2.1 installed and import-verified; full pass-parity requires a dedicated pass mapping for this gate set.
- Hardware metrics: no authenticated provider available in this environment (explicitly logged as unavailable).

The main open lever is NISQ: the exact engine is Clifford-only, and the numeric NISQ pipeline still trails the paper (159 vs. 107) for database-memory reasons documented in §4. Larger-memory machines and the `--deep` graphs are the expected path to narrowing it.

Reproducibility note

All outputs cited in this report are generated by scripts in `scripts/` and committed under `results/`, `figures/`, `report/`, and `future_work/`. Figures regenerate with `PYTHONPATH=src`

`python scripts/generate_figures.py`; the comparison benchmark reruns with `scripts/benchmark_compari` (see README).